

Scanner Map Contributor Guide

Everything a new contributor needs to get Claude Code set up, connect to the shared codebase, and start shipping features that instantly reach all users.

PROJECT	VERSION	AUDIENCE
Invincible.Inc Scanner Map	v0.4 · March 2026	Trusted Contributors

TOOLING
Claude Code CLI + GitHub



CONTENTS

1. [Overview — How the collaboration works](#)
2. [Prerequisites — What you need installed](#)
3. [GitHub Setup — Cloning and branch strategy](#)
4. [Claude Code Setup — Installing and configuring the CLI](#)
 - [Installing Claude Code](#)
 - [Authenticating](#)
 - [Pointing Claude at the project](#)
5. [Daily Workflow — Pull, build, contribute, push](#)
6. [Project Structure — What's where](#)
7. [Backend — FastAPI, SQLite, scanner modules](#)
8. [Frontend — React, Vite, Leaflet](#)
9. [How updates reach all users](#)
10. [Contribution rules and conventions](#)

1 Overview — How the collaboration works

The Scanner Map is a full-stack local application: a FastAPI Python backend that runs on a Windows machine and a React frontend that any device on the same network (or over the internet, if the host is port-forwarded) can access in a browser.

Both contributors write code using **Claude Code** — Anthropic's AI coding CLI — and push their changes to a shared **GitHub repository**. When a user's backend restarts (manually or via the dev panel's git-pull button), it pulls the latest code from GitHub. The frontend is served as a static build bundled inside the backend, so every user's browser gets the new version on next page load.



The user experience: A user on the street opens their browser, the app loads, and they automatically get whatever version is currently deployed on the host machine. There is no app store, no update prompt, no version mismatch. The host operator updates by running `git pull` and restarting the backend.

As a contributor, your role is to write code locally (or let Claude write it), test it, then push to the shared repo on GitHub. The operator (owner of the running machine) then pulls your changes and restarts the backend. Your feature is now live for all users.

2 Prerequisites

TOOL	VERSION / NOTES	INSTALL COMMAND
Node.js	18 or 20 LTS	<code>winget install OpenJS.NodeJS.LTS</code>
Python	3.11 or 3.12	<code>winget install Python.Python.3.12</code>
Git	Any recent version	<code>winget install Git.Git</code>
Claude Code CLI	Latest	<code>npm install -g @anthropic-ai/claude-code</code>
GitHub account	With repo access granted by owner	<code>github.com/signup</code>
VS Code (optional)	For viewing diffs	<code>winget install Microsoft.VisualStudioCode</code>

 **Windows only for hardware scanning.** The backend's WiFi scanner uses `netsh wlan show networks` and Scapy with Npcap — both Windows-specific. You can develop and test logic on macOS/Linux, but hardware scanning only works on Windows.

3 GitHub Setup

Getting access

The repo owner will grant you **Write** access to the repository. You'll receive an email invitation. Accept it, then you can clone and push directly — no fork required.

Clone the repo

```
# Clone to your local machine
git clone https://github.com/<owner>/scanner-map.git
cd scanner-map

# Verify you can see the main branch
git log --oneline -5
```

Branch strategy

Work on a feature branch, not directly on `main`. This prevents your in-progress changes from breaking the running instance before they're tested.

```
# Create a branch for your feature
git checkout -b feature/my-feature-name

# When ready to ship, push and ask the owner to merge (or merge yourself if you have permission)
git push origin feature/my-feature-name
```



Branch naming convention: `feature/description` for new features, `fix/description` for bug fixes, `refactor/description` for code cleanup. Keep names lowercase with hyphens.

Before every session — always pull first

```
git checkout main
git pull origin main
git checkout -b feature/your-new-feature
```

4 Claude Code Setup

4a. Installing Claude Code

```
# Install globally via npm
npm install -g @anthropic-ai/claude-code

# Verify installation
claude --version
```

4b. Authenticating

Claude Code uses your Anthropic account. Run the login command and follow the browser prompt:

```
claude login
```

This opens a browser window to authenticate with your Anthropic account. You need an active Claude Pro or API subscription. If you don't have one, go to **anthropic.com** and create an account — Pro plan (\$20/mo) includes Claude Code access.

4c. Pointing Claude at the project

Navigate to the project root in your terminal, then launch Claude Code. It automatically reads the `CLAUDE.md` file (if present) for project-specific instructions.

```
cd C:\path\to\scanner-map
claude
```

Once in the Claude Code prompt, you can describe what you want to build and Claude will read the relevant files, write the code, and verify it builds. Example session:

**Example Claude Code session:**

You: "Add a new endpoint `GET /scan/wifi-history` that returns the last 1000 WiFi observations grouped by SSID with signal trend over time."

Claude: Reads `backend/src/app/api/scan.py`, reads the DB schema, writes the endpoint, verifies it with a quick build check, confirms done.

Setting your identity in git

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

5 Daily Workflow

1 Pull the latest code

Always start by syncing with main to avoid merge conflicts later.

2 Create a feature branch

```
git checkout -b feature/what-youre-building
```

3 Open Claude Code in the project root

Run `claude` in the `scanner-map/` directory. Describe your feature.

4 Build and test

For frontend changes: `cd frontend && npm run build`. For backend: run the backend server and test the endpoint in a browser or Postman.

5 Commit your changes

Stage specific files: `git add path/to/file`. Never use `git add .` blindly — avoid committing build artifacts or secrets.

6 Push and notify

`git push origin feature/your-branch`. Message the owner to review and merge, or merge to main yourself if you have permission and tested thoroughly.

7 Owner deploys

The running host does `git pull && npm run build` (frontend) and restarts the backend. Users get the update on next page refresh.

The full build sequence

```
# Backend (Python) — no build step needed, just restart
cd scanner-map/backend
pip install -r requirements.txt          # only needed if you added packages
python src/main.py                      # or however you start it

# Frontend (React → static files)
cd scanner-map/frontend
npm install                             # only needed if you added packages
npm run build                           # outputs to dist/
# The backend serves dist/ automatically
```

6 Project Structure

```

scanner-map/
├── backend/
│   └── src/
│       └── app/
│           ├── api/           # FastAPI route handlers
│           │   ├── scan.py    # /scan/* endpoints
│           │   ├── users.py   # /users/* endpoints
│           │   ├── gps.py     # /gps/* endpoints
│           │   └── ...
│           ├── ingest/       # Scanner modules (wifi, ble, cell, etc.)
│           │   ├── wifi_scanner.py
│           │   ├── ble_scanner.py
│           │   ├── aggressive_scanner.py
│           │   ├── network_discovery.py
│           │   └── cell_scanner.py
│           ├── processing/   # Data aggregation and heatmap logic
│           ├── db/          # SQLite connection and schema
│           ├── core/        # Config, allowlist, constants
│           └── main.py       # FastAPI app, router registration
├── frontend/
│   └── src/
│       └── components/
│           ├── MapView.jsx   # Main map + heatmap
│           ├── Sidebar.jsx   # User-facing scan controls
│           ├── DevPanel.jsx  # /#dev - operator console
│           └── ...
├── explainer/
│   ├── index.html           # Marketing / explanation website
│   └── contributor-guide.html # This document
└── CLAUDE.md                # Claude Code project instructions

```

7 Backend — FastAPI + SQLite

Adding a new API endpoint

Every new route group gets its own file in `backend/src/app/api/`. Register it in `main.py` with a prefix.

```

# 1. Create backend/src/app/api/myfeature.py
from fastapi import APIRouter
router = APIRouter()

@router.get("/hello")
def hello():
    return {"message": "world"}

# 2. In main.py, add:

```

```
from app.api import myfeature
app.include_router(myfeature.router, prefix="/myfeature")
```

Database access

```
from app.db.database import get_db

conn = get_db()
rows = conn.execute("SELECT * FROM raw_observations LIMIT 10").fetchall()
# rows are sqlite3.Row objects – access as dict: row["column_name"]
```

Key DB tables

TABLE	PURPOSE	KEY COLUMNS
<code>raw_observations</code>	Every scanner hit	target_key, source, ts_ms, rssi, lat, lon, meta_json
<code>encounters</code>	Aggregated device encounters	target_key, first_ts_ms, peak_ts_ms, hit_count, peak_rssi
<code>heatmap_cells</code>	Grid heat density	lat_bucket, lon_bucket, hit_count, last_ts_ms
<code>route_points</code>	GPS trace	ts_ms, lat, lon, speed_mps, heading

8 Frontend — React + Vite

Adding a new UI component

```
// frontend/src/components/MyWidget.jsx
import React, { useState, useEffect } from 'react'

export default function MyWidget() {
  const [data, setData] = useState(null)
  useEffect(() => {
    fetch('/myfeature/hello').then(r => r.json()).then(setData)
  }, [])
  return <div>{data?.message}</div>
}
```

Dev server (hot reload)

```
cd frontend
npm run dev
```



```
# App served at http://localhost:5173
# Backend must be running separately at http://localhost:8000
# API calls in dev are proxied automatically via vite.config.js
```

Building for production

```
npm run build
# Outputs to frontend/dist/
# Backend's static file handler serves this directory
```

⚠ **Always run `npm run build` before committing frontend changes.** The backend serves the `dist/` directory. If you only save `.jsx` files without building, users won't see your changes.

9 How updates reach all users

The deployment chain is intentionally simple:

```
You push to GitHub
↓
Owner runs: git pull && cd frontend && npm run build
↓
Backend restarts (serves new dist/ files)
↓
User refreshes browser → gets new frontend
↓
User's API calls hit new backend endpoints automatically
```

There is no container, no CI/CD pipeline, no cloud deployment required. Everything runs on the operator's local machine. The "deployment" is literally `git pull` and a backend restart.

Auto-pull button (in Dev Panel → Lab)

The operator can also trigger a git pull + frontend rebuild from the Dev Panel without touching a terminal. This makes updates a one-click operation during field use.

What users see

- Frontend updates: visible immediately on next page refresh (browser loads new JS/CSS bundle)
- New API endpoints: available as soon as the backend restarts
- Database schema changes: applied automatically if the backend runs migrations on startup
- No forced refresh, no service worker cache issues — Vite hashes bundle filenames so old files never get served from browser cache

10 Contribution rules and conventions

Code rules

- **Backend endpoints:** always wrap in try/except and return graceful JSON on failure
- **Database queries:** always use parameterized queries — never string-format SQL
- **Frontend:** no new npm dependencies without discussion — bundle size matters on mobile
- **No secrets in code:** Twilio keys, API tokens → `runtime_settings.json` only
- **Build before committing:** run `npm run build` and verify it passes

What Claude Code will and won't help with

Claude Code is instructed to refuse features in specific categories. These are hard-coded refusals — providing more context or reframing the request won't change the answer:

- Deauthentication / disassociation frame injection
- Fake beacon frame broadcasting
- BLE pairing attacks or DoS
- ARP poisoning / man-in-the-middle
- Intercepting or redirecting Axon streams
- Any active frame injection on networks you don't own

Everything else — new scanner modules, UI features, data analysis, GPS processing, social features, settings — Claude will help build.

Getting unstuck

- Read the error message carefully — Python stack traces and Vite build errors are usually self-explanatory
- Ask Claude: paste the error and say "what's wrong and how do I fix it"
- Check `backend/src/app/main.py` to make sure your router is registered
- Check the browser console (F12) for frontend errors
- If the backend won't start, run it with `python -u src/main.py` and read the traceback



You're ready. Clone the repo, run `claude` in the project root, and tell it what you want to build. It will read the codebase, write the code, and verify it compiles. Push your branch and notify the owner. That's the whole workflow.

Invincible.Inc · Scanner Map Contributor Guide · v0.4 · March 2026

For access issues, contact the repo owner on GitHub.